Guide du Projet — SMSTSIKA

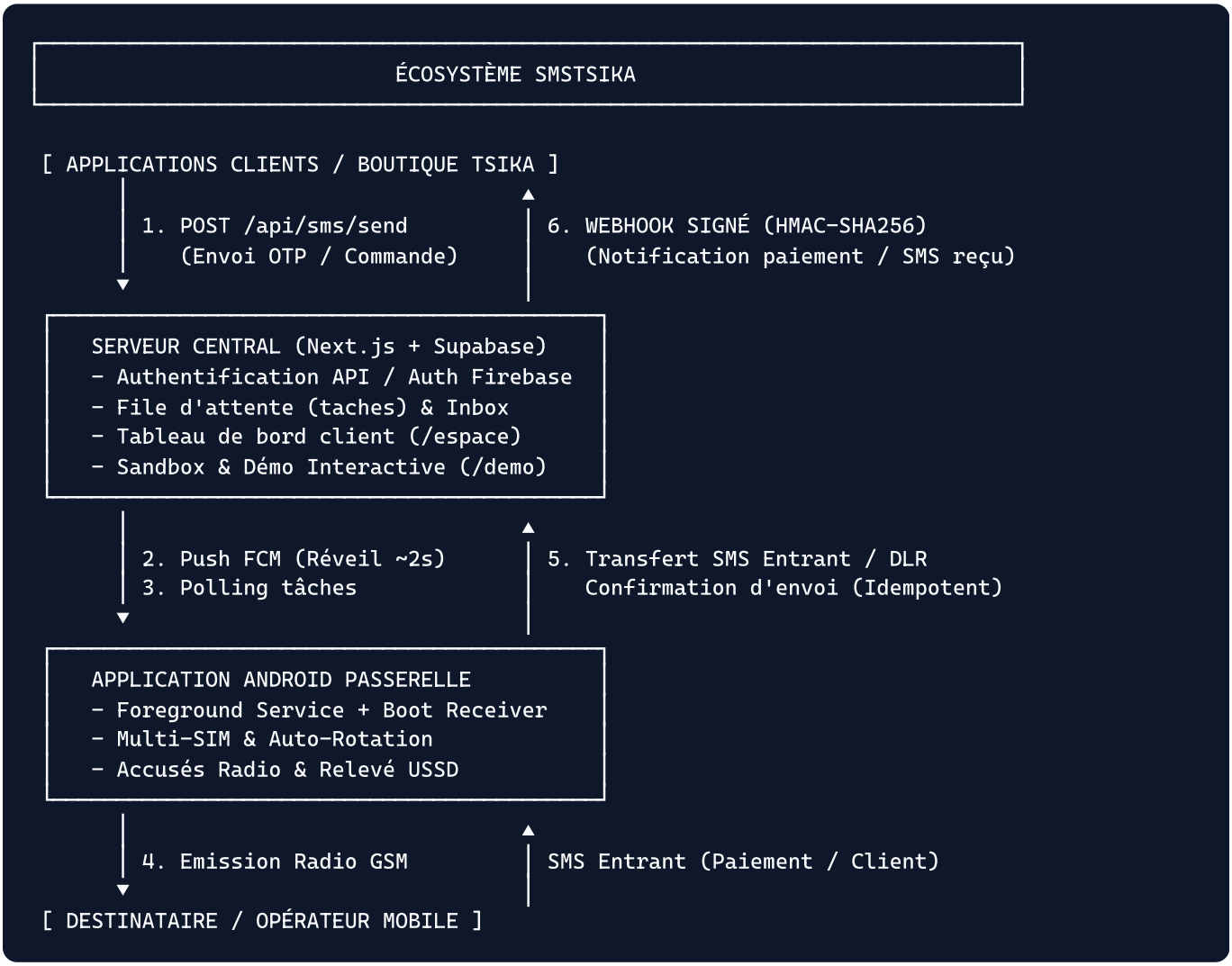
Spécifications Techniques, Architecture & Guide du Développeur (Version 3.0 Production)

1. C'est quoi ce projet, en simple ?

SMSTSIKA est un système complet de **passerelle SMS bidirectionnelle (Envoi & Réception)** et de **Webhooks en temps réel**. Il permet à n'importe quelle application (site e-commerce, application web, logiciel de caisse, CRM, bot) d'émettre et de recevoir des SMS sans dépendre d'APIs payantes coûteuses comme Twilio.

Au lieu de cela, le système s'appuie sur des **smartphones Android dédiés avec de vraies cartes SIM**. Chaque téléphone fait tourner l'application **SMSTSIKA Android** en arrière-plan (service permanent *Foreground Service*).

Le principe en un schéma d'architecture :



2. Le vocabulaire à connaître (Mise à jour v3.0)

Terme	Explication simple
API REST	Interface permettant aux applications externes d'envoyer des SMS (`POST /api/sms/send`).
File d'attente (Queue)	Gestion ordonnée des SMS à envoyer pour lisser le trafic et respecter les limites.
Push FCM (Firebase)	Notification push silencieuse envoyée au téléphone pour interrompre l'attente et traiter le SMS en ~2s.
Foreground Service	Service Android avec notification permanente (`SMSTSIKA actif`) empêchant le système de tuer l'application.
Inbound / SMS Entrant	Capacité du téléphone à intercepter les SMS reçus sur la SIM (ex: avis Mobile Money) et les transmettre au serveur.
Webhook Signé (HMAC)	Notification HTTP automatique transmise au site client lors d'un SMS entrant, sécurisée par signature cryptographique HMAC-SHA256.
Accusé Réseau (DLR)	Confirmation réseau niveau opérateur (`LIVRE` / `ECHEC_LIVRAISON`) via `deliveryIntent` .
Multi-SIM Auto-Rotation	Bascule automatique de carte SIM tous les 10 SMS pour équilibrer la charge et éviter le bannissement opérateur.
Code USSD	Commande réseau (ex: `*111#`) exécutée automatiquement par l'app pour consulter le solde de la SIM (`solde_sim`).
Idempotence	Garantie qu'une panne réseau ne provoque jamais un double envoi de SMS au destinataire.
Appairage QR Code	Liaison instantanée en 1 clic entre le téléphone Android et le tableau de bord web.

3. Les trois briques de l'architecture

Le projet SMSTSIKA se compose de **trois briques autonomes et complémentaires** :

```
SmsGateway/
├── 1. app/                  👉 Application Android Passerelle (Kotlin / Jetpack Compose)
├── 2. sms-gateway-server/   👉 Serveur Central & Console Web (Next.js / Supabase / Firebase)
└── 3. boutique-tsika/       👉 Site Client Démo E-Commerce (Vercel)
```

A. Le Serveur Central (`sms-gateway-server`)

- **Base de données PostgreSQL (Supabase)** : Tables `taches` , `reponses` , `appareils` , `applications` , `notifications` , `parametres` .

- **Securité & RLS** : Authentification multi-tenant (Firebase Auth + Supabase Row Level Security).
- **Console Client (`/espace`)** : Dashboard d'administration, clés API, gestion des appareils, boîte de réception, webhooks, et **Graphique de Consommation Quotidienne en bâtons bleus**.
- **Bac à Sable Interactive (`/demo`)** :
  - **Module 1 (OTP)** : Démo live d'envoi et validation de code OTP.
  - **Module 2 (Paiement)** : Démo live de commande e-commerce avec détection automatique de paiement par SMS entrant.

B. L'Application Android (`app`)

1. **Authentification & Appairage** : Connexion Firebase + Scan de QR Code.
2. **Service Permanent** : Execution continue en arrière-plan (`ServicePasserelleSms`).
3. **Moteur d'Émission** : Envoi de SMS avec `SmsManager` (single & multipart) + validation par `PendingIntent` .
4. **Moteur de Réception** : Capture des SMS reçus (`RecepteurSms`) et transfert vers `/api/devices/[id]/inbox` .
5. **Gestionnaire Multi-SIM** : Rotation automatique ou sélection manuelle de la carte SIM.
6. **Interface Jetpack Compose** : Écrans *Statut, Tâches, Journal, Diagnostic, Stats, Paramètres, Scan QR*.

C. Le Site Client Démo (`boutique-tsika`)

- Site marchand autonome déployé séparément sur Vercel.
- Illustre l'utilisation réelle de l'API d'envoi SMSTSIKA et la réception des Webhooks signés HMAC pour marquer les commandes comme **"Payées"**.

4. Statuts et Cycles de Vie

Statuts d'un Appareil (Téléphone)

- `EN_LIGNE / ONLINE` : Le téléphone est connecté, actif et prêt à traiter des SMS.
- `HORS_LIGNE / OFFLINE` : Le téléphone est déconnecté (perte réseau ou arrêt du service).
- `DESACTIVE / DISABLED` : Appareil désactivé par l'administrateur.

Statuts d'une Tâche SMS (Envoi Sortant)

```
[ EN_ATTENTE ] ⟶ [ RECLAME ] ⟶ [ ENVOYE ] ⟶ [ LIVRE ] (Accusé DLR Réseau)
                                  └⟶ [ ECHOUE ]
```

Statuts d'un SMS Entrant / Webhook (Réception)

```
[ RECU_SIM ] ⟶ [ TRANSFAIRE_SERVEUR ] ⟶ [ ENFILER_WEBHOOK ] ⟶ [ LIVRE_AU_CLIENT (200 OK) ]
```

5. Les Pièges Techniques Résolus (Best Practices Production)

🛡️ Piège 1 : Arrêt de l'application par l'économe de batterie Android

Solution mise en place : Utilisation d'un **Foreground Service** avec canal de notification prioritaire + relance automatique au démarrage (`BOOT_COMPLETED`).

🛡️ Piège 2 : Blocage de la carte SIM par l'opérateur (Anti-Spam)

Solution mise en place :

- **Rotation automatique Multi-SIM** tous les 10 SMS.
- **Pacing / Temporisation** entre les envois consécutifs.
- Suivi des quotas horaires et mensuels.

🛡️ Piège 3 : Risque de double envoi en cas de coupure réseau

Solution mise en place : Réclamation préalable (`RECLAME`) avant tout envoi SMS. Le SMS n'est émis qu'une seule fois. Si la connexion coupe après l'envoi, seule la confirmation HTTP est sauvegardée localement et rejouée dès la reconnexion (`confirmerAvecReessai`).

🛡️ Piège 4 : Restrictions Google Play Store sur les permissions SMS

Solution mise en place : Déploiement direct via fichier **APK** installable sur téléphones dédiés.

6. Livrables du Projet SMSTSIKA v3.0

1. **Serveur Central Next.js + Supabase** (Prêt pour Vercel / Production).
2. **API REST d'envoi de SMS** documentée et sécurisée par clés API.
3. **Application Android APK** téléchargeable avec support Multi-SIM, FCM, DLR et USSD.
4. **Moteur de Webhooks Réseau** avec signatures de sécurité HMAC-SHA256.
5. **Dashboard Web Client (`/espace`)** avec graphique dynamique de consommation quotidienne.
6. **Bac à Sable / Démo Interactive (`/demo`)** pour les démonstrations OTP & Paiement.
7. **Boutique Client Démo (`boutique-tsika`)** montrant la boucle d'intégration complète.